ReviewPilot - Week 1 Integration Guide

For: Rahul Gupta (guptarahul07@gmail.com)
Timeline: 5-6 hours this week
Goal: Add Contact page, Analytics tracking, and Google Cloud setup

---

✅ Your Details (Already Updated in Files)

- **Email:** guptarahul07@gmail.com
- **WhatsApp:** +91 9810026181
- **Contact Form:** Enabled ✓
- **Admin Dashboard:** Simple version ✓

All files have been pre-configured with your contact info. Just copy-paste!

📂 File Organization

Here's where each file goes in your project:

```
reviewpilot/
├── backend/
|   ├── utils/
|   |   └── analytics.js          ← NEW (copy from downloads)
|   ├── routes/
|   |   └── analytics.js          ← NEW (copy from downloads)
|   └── index.js                  ← UPDATE (add imports)
|
└── frontend/
    └── src/
        ├── components/
        |   ├── FeedbackWidget.jsx     ← NEW
        |   └── FeedbackWidget.css     ← NEW
        ├── pages/
        |   ├── Contact.jsx            ← NEW
        |   ├── Contact.css            ← NEW
        |   ├── AdminDashboard.jsx     ← NEW
        |   └── AdminDashboard.css     ← NEW
        └── App.jsx                    ← UPDATE (add routes)
```

🚀 Step-by-Step Integration (Do in Order)

STEP 1: Backend - Add Analytics System (30 minutes)

1.1 Create folders if they don't exist:

```bash
cd backend
mkdir -p utils routes
```

1.2 Copy analytics files:

```bash
# Copy the analytics.js file you downloaded to:
cp ~/Downloads/analytics.js backend/utils/analytics.js

# Copy the analytics-routes.js file you downloaded to:
cp ~/Downloads/analytics-routes.js backend/routes/analytics.js
```

1.3 Update `backend/index.js`:

Add these imports at the top:

```javascript
import analyticsRoutes from './routes/analytics.js';
import { trackEvent } from './utils/analytics.js';
```

Add this route (after your other app.use() calls):

```javascript
// Analytics routes
app.use('/api', analyticsRoutes);
```

Add tracking to your existing sync route:

```javascript
```

```
// Find your existing /api/reviews/sync route and add this line at the end:
app.post("/api/reviews/sync", async (req, res) => {
  // ... your existing code ...

  // ADD THIS LINE before res.json():
  await trackEvent(uid, 'review_synced', { reviewCount: results.length });

  res.json({ reviews: results });
});
```

1.4 Test backend:

```bash
cd backend
npm start

# You should see: "Backend running on http://localhost:5000"
# No errors = good to go!
```

STEP 2: Frontend - Add Contact Page (20 minutes)

2.1 Copy Contact files:

```bash
cd frontend/src/pages

# Copy the Contact.jsx and Contact.css files you downloaded:
cp ~/Downloads/Contact.jsx ./Contact.jsx
cp ~/Downloads/Contact.css ./Contact.css
```

2.2 Update `frontend/src/App.jsx`:

Add import at top:

```javascript
import Contact from './pages/Contact';
```

Add route in your Routes section (under public routes):

```javascript
<Route element={<PublicLayout />}>
  <Route path="/" element={<Home />} />
  <Route path="/pricing" element={<Pricing />} />
  <Route path="/contact" element={<Contact />} />  {/* ← ADD THIS */}
</Route>
```

2.3 Add Contact link to navigation:

Update `frontend/src/components/layout/PublicLayout.jsx` (or wherever your nav is):

```javascript
<nav>
  <Link to="/">Home</Link>
  <Link to="/pricing">Pricing</Link>
  <Link to="/contact">Contact</Link>  {/* ← ADD THIS */}
  <Link to="/login">Login</Link>
</nav>
```

2.4 Test Contact page:

```bash
cd frontend
npm run dev

# Visit: http://localhost:5173/contact
# You should see your email (guptarahul07@gmail.com) and WhatsApp
# Click the email link → opens Gmail
# Click WhatsApp → opens WhatsApp Web with pre-filled message
```

STEP 3: Frontend - Add Admin Dashboard (20 minutes)

3.1 Copy Admin Dashboard files:

```bash
```

```bash
cd frontend/src/pages

cp ~/Downloads/AdminDashboard.jsx ./AdminDashboard.jsx
cp ~/Downloads/AdminDashboard.css ./AdminDashboard.css
```

3.2 Update `frontend/src/App.jsx`:

Add import:

```javascript
import AdminDashboard from './pages/AdminDashboard';
```

Add route (under protected routes):

```javascript
<Route element={<ProtectedRoute><AppLayout /></ProtectedRoute>}>
  <Route path="/connect" element={<ConnectGoogle />} />
  <Route path="/reviews" element={<ReviewsInbox />} />
  <Route path="/settings" element={<Settings />} />
  <Route path="/admin" element={<AdminDashboard />} />  {/* ← ADD THIS */}
</Route>
```

3.3 Test Admin Dashboard:

```bash
# Visit: http://localhost:5173/admin
# (Make sure you're logged in with guptarahul07@gmail.com)

# You should see:
# - Total Users: 0 (or 1 if you already created an account)
# - Empty feedback list
# - Empty metrics
```

This is normal! Metrics will populate as you use the app and track events.

STEP 4: Frontend - Add Feedback Widget (15 minutes)

4.1 Copy Feedback Widget files:

```bash
cd frontend/src/components

cp ~/Downloads/FeedbackWidget.jsx ./FeedbackWidget.jsx
cp ~/Downloads/FeedbackWidget.css ./FeedbackWidget.css
```

4.2 Add to Settings page:

Update `frontend/src/pages/Settings.jsx`:

Add import at top:

```javascript
import FeedbackWidget from '../components/FeedbackWidget';
```

Add widget in the header section:

```javascript
<div className="settings__header">
  <h1 className="settings__title">Settings</h1>
  <FeedbackWidget />  {/* ← ADD THIS */}
</div>
```

4.3 Test Feedback Widget:

```bash
# Visit: http://localhost:5173/settings
# Click the "💬 Feedback" button
# Fill out the form:
#   - Rate 5 stars
#   - Select a feature: "AI reply generation"
#   - Comment: "Testing feedback system!"
# Click "Send Feedback"
# Should show success message ✓
```

4.4 Verify in Firestore:

1. Go to Firebase Console: https://console.firebase.google.com
2. Select your project
3. Click "Firestore Database"

4. You should see a new collection: `feedback`
5. Inside, your test feedback should appear!

STEP 5: Google Cloud Setup (1-2 hours)

This is the most important step. Follow the guide carefully:

5.1 Open the Google Cloud Setup Guide:

- File: `Google_Cloud_Setup_Guide.md` (you downloaded this)
- Follow every step exactly as written
- Takes 15-20 minutes

5.2 What you'll do:

1. Create Google Cloud Project "ReviewPilot"
2. Enable 2 APIs (My Business APIs)
3. Create OAuth consent screen
4. Get OAuth credentials (Client ID + Secret)

5.3 Save credentials to `.env`:

Update `backend/.env`:

```
CLAUDE_API_KEY=sk-ant-api03-V_osjkJ0h4... (your existing key)

# Add these new lines:
GOOGLE_CLIENT_ID=123456-abcdef.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-aBcDeFgHiJkLmN
GOOGLE_REDIRECT_URI=http://localhost:5173/auth/google/callback
```

5.4 IMPORTANT Security:

Make sure `.env` is in `.gitignore`:

```bash
```

```
# Check:
cat backend/.gitignore

# Should include:
.env
node_modules
```

If not, add it:

```bash
echo ".env" >> backend/.gitignore
```

✅ Testing Checklist - Do This After Integration

Run through this checklist to make sure everything works:

Backend Tests:

- [ ] `npm start` runs without errors
- [ ] Visit `http://localhost:5000` shows "Backend running 🚀"
- [ ] No console errors about missing imports

Frontend Tests:

- [ ] `npm run dev` runs without errors
- [ ] Visit `/contact` → see your email and WhatsApp
- [ ] Click email link → opens Gmail compose
- [ ] Click WhatsApp → opens WhatsApp Web
- [ ] Fill contact form → submits successfully
- [ ] Visit `/admin` → dashboard loads (empty is ok)
- [ ] Visit `/settings` → see feedback button
- [ ] Click feedback → modal opens
- [ ] Submit feedback → success message

Firestore Tests:

- [ ] Open Firebase Console
- [ ] Check `feedback` collection exists
- [ ] See your test feedback entry
- [ ] Check `events` collection (will populate when you track events)

Navigation Tests:

- [ ] Contact link appears in nav
- [ ] Clicking Contact takes you to `/contact`
- [ ] No 404 errors

📊 What Gets Tracked Automatically

Once you integrate analytics, these events auto-track:

Event	When It Happens	What Gets Saved
`user_signed_up`	New user registers	User ID, timestamp
`google_connected`	User connects GBP	User ID, timestamp, business name
`review_synced`	Reviews fetched	User ID, review count
`reply_generated`	AI generates reply	User ID, review ID
`reply_posted`	Reply posted to Google	User ID, review ID, auto/manual
`sentiment_flagged`	Review flagged	User ID, review ID

To view metrics:

- Visit `/admin`
- You'll see totals, active users, conversion rates
- Export data as JSON for weekly analysis

🎯 What You'll Have After Week 1

After completing all 5 steps:

✅ Contact Page

- Users can email you (guptarahul07@gmail.com)
- Users can WhatsApp you (+91 9810026181)
- Contact form submissions saved (you'll build email later)

✅ Analytics System

- Track every user action
- Admin dashboard to view metrics
- Export data for analysis

✅ Feedback Collection

- Users can rate the app (1-5 stars)
- Users can report issues or requests
- All feedback saved in Firestore

✅ Google Cloud Ready

- OAuth credentials obtained
- Ready to build Google API integration next week

🐛 Troubleshooting Common Issues

Issue 1: "Cannot find module './routes/analytics.js'"

Fix: Make sure you created the `backend/routes` folder and copied the file there.

Issue 2: "Failed to fetch analytics"

Fix: Check that you're logged in with `guptarahul07@gmail.com` (the admin email).

Issue 3: Feedback widget doesn't show

Fix: Make sure you imported it AND added `<FeedbackWidget />` to the JSX.

Issue 4: Contact page email link doesn't work

Fix: Make sure there are no spaces in the email: `guptarahul07@gmail.com`

Issue 5: WhatsApp link doesn't work

Fix: Number format must be `+919810026181` (no spaces, starts with +91)

📝 What to Do This Week (5-6 Hours)

Session 1 (2 hours):

- [ ] Integrate backend analytics (Step 1)
- [ ] Add Contact page (Step 2)
- [ ] Test both

Session 2 (2 hours):

- [ ] Add Admin Dashboard (Step 3)
- [ ] Add Feedback Widget (Step 4)
- [ ] Test everything

Session 3 (1-2 hours):

- [ ] Google Cloud setup (Step 5)
- [ ] Save credentials to `.env`
- [ ] Run full testing checklist

End of Week 1: You'll have contact page, analytics, and Google OAuth ready!

🚀 Next Week (Week 2)

Once you finish this week, I'll help you build:

1. **Complete Google OAuth Flow**
   - Backend routes for `/auth/google/connect` and `/auth/google/callback`
   - Frontend "Connect Google Business" button
   - Token storage in Firestore
2. **Fetch Real Reviews**
   - Call Google My Business API
   - Store reviews in Firestore
   - Display in ReviewsInbox
3. **Post Replies to Google**
   - Build `/api/reviews/post` endpoint
   - Actually post to Google (not just mark as posted)
   - Handle errors and rate limits

By end of Week 2: Your app will work with real Google data! 🎉

💬 Questions?

If you get stuck:

1. Check the Troubleshooting section above
2. Google the exact error message
3. Ask me and share the error - I'll help debug!

You're ready to start! Begin with Step 1 (Backend Analytics) and work through in order. 🔥